

Interface in Java

An interface in Java is a blueprint of a class. It is used to achieve abstraction and multiple inheritance

An interface contains:

- Abstract methods (methods without body)
- Constants (variables that are public static final by default)

A class implements an interface and provides the implementation of its methods.

1. Syntax of Interface

```
interface InterfaceName
{
    void method1();
    void method2();
}

class ClassName implements InterfaceName
{
    public void method1()
    {
        // implementation
    }

    public void method2()
    {
        // implementation
    }
}
```

2. Simple Example of Interface

```
interface Animal
{
    void sound();
}

class Dog implements Animal
{
    public void sound()
    {
        System.out.println("Dog barks");
    }
}

class Main
{
    public static void main(String[] args)
    {
        Dog d = new Dog(); d.sound();
    }
}
```

Output: Dog barks

3. Interface with Multiple Methods

```
interface Shape
{
    void area(); void
    display();
}

class Circle implements Shape
{
    public void area()
    {
        System.out.println("Area of Circle");
    }
}
```

```

    }

    public void display()
    {
        System.out.println("This is a Circle");
    }
}

class Main
{
    public static void main(String[] args)
    {
        Circle c = new Circle(); c.area();
        c.display();
    }
}

```

4. Multiple Inheritance Using Interface

```

interface A
{
    void showA();
}

interface B
{
    void showB();
}

class Test implements A, B
{
    public void showA()
    {
        System.out.println("Interface A method");
    }

    public void showB()
    {
        System.out.println("Interface B method");
    }
}

class Main
{
    public static void main(String[] args)
    {
        Test t = new Test(); t.showA();
        t.showB();
    }
}

```

Important Points About Interface

1. All methods are public and abstract by default.
2. All variables are public, static, and final.
3. An interface cannot be instantiated.
4. A class can implement multiple interfaces.
5. Interfaces help achieve abstraction.

Abstract Class in Java

An abstract class in Java is a class that cannot be instantiated (we cannot create objects of it). It is used to achieve partial abstraction.

An abstract class may contain:

- Abstract methods (methods without body)
- Normal methods (methods with body)
- Variables
- Constructors

A class that inherits an abstract class must implement all its abstract methods.

Syntax of Abstract Class

```
abstract class ClassName
{
    abstract void method1();

    void method2()
    {
        System.out.println("This is a normal method");
    }
}

class ChildClass extends ClassName
{
    void method1()
    {
        System.out.println("Abstract method implemented");
    }
}
```

Example of Abstract Class

```
abstract class Animal
{
    abstract void sound();
}

class Dog extends Animal
{
    void sound()
    {
        System.out.println("Dog barks");
    }
}

class Main
{
    public static void main(String[] args)
    {
        Dog d = new Dog(); d.sound();
    }
}
```

Output: Dog barks

Abstract Class with Abstract and Normal Method

```
abstract class Shape
{
    abstract void area();

    void display()
    {
        System.out.println("This is Shape class");
    }
}
```

```
class Circle extends Shape
{
    void area()
    {
        System.out.println("Area of Circle");
    }
}

class Main
{
    public static void main(String[] args)
    {
        Circle c = new Circle();
        c.display();
        c.area();
    }
}
```

Important Points About Abstract Class

1. An abstract class cannot be instantiated.
2. It can contain both abstract and non-abstract methods.
3. It can contain constructors and variables.
4. A class can extend only one abstract class.
5. If a class does not implement all abstract methods, it must also be declared abstract.